

Chapter 14: Newspaper App

Articles App - Step by Step

This guide converts the Chapter 14 Articles App section into a practical checklist. It starts by creating an **articles** app, adding an **Article** model, registering it in Django admin, and improving the admin list display.

Important: The book's Chapter 14 assumes a larger Newspaper project with accounts, pages, and user authentication. If you are continuing from the current blog project, use only the apps you actually have installed.

Step 1: Go To The Main Project Folder

Open PowerShell and move into the project folder.

```
cd "C:\Users\ffaay\Documents\django-blog-from-start"  
.\.venv\Scripts\Activate.ps1
```

You should see **(.venv)** at the start of the terminal line.

Step 2: Create The Articles App

```
python manage.py startapp articles
```

After this command, Django creates this folder:

```
articles/  
■■■ admin.py  
■■■ apps.py  
■■■ models.py  
■■■ tests.py  
■■■ views.py  
■■■ migrations/
```

Step 3: Add The App To Settings

Open **django_project/settings.py** and add **articles** to **INSTALLED_APPS**.

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "blog",  
    "articles",  
]
```

Do not add **crispy_forms**, **accounts**, or **pages** unless those apps already exist in your project.

Step 4: Set The Time Zone

In `django_project/settings.py`, find `TIME_ZONE` and set it for India:

```
TIME_ZONE = "Asia/Kolkata"
```

Step 5: Create The Article Model

Open `articles/models.py` and replace it with this code:

```
from django.conf import settings
from django.db import models
from django.urls import reverse

class Article(models.Model):
    title = models.CharField(max_length=255)
    body = models.TextField()
    date = models.DateTimeField(auto_now_add=True)
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )

    def __str__(self):
        return self.title

    def get_absolute_url(self):
        return reverse("article_detail", kwargs={"pk": self.pk})
```

The model contains these fields:

- **title:** article headline
- **body:** article content
- **date:** automatic timestamp
- **author:** user who wrote the article

Step 6: Make And Apply Migrations

Run these commands to create and apply the database table:

```
python manage.py makemigrations articles
python manage.py migrate
```

Step 7: Register Article In Admin

Open `articles/admin.py` and add:

```
from django.contrib import admin

from .models import Article

admin.site.register(Article)
```

Step 8: Start The Server

```
python manage.py runserver
```

Open the admin page:

```
http://127.0.0.1:8000/admin/
```

If you do not have a superuser yet, stop the server with Ctrl + C and run **python manage.py createsuperuser**.

Step 9: Add Articles In Admin

In Django admin, click **Articles**, then click **+ Add**.

Create sample articles such as:

```
Title: World News
Body: Some things happened around the world.
Author: your superuser

Title: Local News
Body: Not much happened locally to be honest.
Author: your superuser
```

Step 10: Improve The Admin List Display

Replace `articles/admin.py` with:

```
from django.contrib import admin

from .models import Article

class ArticleAdmin(admin.ModelAdmin):
    list_display = [
        "title",
        "body",
        "author",
    ]
```

```
admin.site.register(Article, ArticleAdmin)
```

Refresh the admin articles page. You should now see:

```
Title | Body | Author
```

Step 11: Save With Git

```
git status
git add -A
git commit -m "Add articles app and model"
```

Completion Checklist

- Created the articles app.
- Added articles to `INSTALLED_APPS`.
- Set the time zone to Asia/Kolkata.
- Created the Article model.
- Ran migrations.
- Registered Article in Django admin.
- Created sample articles.
- Improved admin list display.
- Saved the work with Git.

Next section: URLs and Views, where articles become visible on the public website.